

The Dangers of Change Approval Processes

This was an interesting topic for me because I work in network operations and I have seen our change management process evolve from a manual review and approval process into a more automated system. Before our current change management system, we held Method of Procedure (MOP) reviews that required approval from the implementer, manager, and director as a way of managing risk before changes were implemented. I agree that having a process to review and approve changes can help prevent mistakes, maintain compliance, and protect production environments. However, I am glad we have moved to a more automated approval process where the system can identify conflicting projects that could negatively impact the network if implemented at the same time. When conflicts are identified, they still require management approval before moving forward. After researching this topic and reviewing the concepts from The DevOps Handbook, I agree that organizations need controls around changes, but those controls should provide confidence and faster feedback rather than create unnecessary delays.

In today's DevOps environment, software development practices focus on smaller, more frequent releases supported by automation, testing, and monitoring. The Atlassian article suggested that change management should focus on making changes safer through automation and planning rather than adding additional approval steps. By incorporating practices such as automated testing, feature flags, and controlled deployments, organizations can reduce risk throughout the development process instead of depending primarily on manual approvals. When organizations continue relying on a manual approval process as the primary method of reducing risk, they can create bottlenecks that slow down innovation and delay feedback.

The LaunchDarkly article did research from the DevOps Research and Assessment (DORA) and indicated that heavyweight change approval processes are associated with lower software delivery performance and higher change failure rates. When teams are unable to deploy smaller changes regularly, they combine multiple features, fixes, and

updates into a single release. Larger releases can make it more difficult to determine deployment failures and increase troubleshooting times.

The PLAT4MATION article indicated organizations can reduce many delays by integrating change management directly into the DevOps pipeline. Instead of requiring developers to leave their normal workflow to complete manual change requests, automation can capture deployment information, support risk-based approvals, and maintain visibility without slowing down the software delivery. This allows organizations to maintain governance while reducing administrative work and improving collaboration between the development and operations teams. This solution is not currently part of my organization's change management process, but it is something I plan to bring up during our developer calls because many of us agree that we spend too much time in operations creating manual change notifications.

Instead of relying only on manual approvals, an organization can improve the approval process through automated testing, security checks, peer reviews, and deployment monitoring. The goal of DevOps is not to eliminate controls; the goal is to replace slow manual controls with faster and more reliable feedback. Automated testing can validate that code changes work as expected before deployment. Monitoring and telemetry can provide immediate feedback after a release. In my role, I have access to a telemetry device we call the Real Time Tool (RTT) so I can see immediate Key Performance Indicators (KPI's) and assess if the change was successful or needs to be rolled back. Peer reviews can also be a good way to improve quality by sharing knowledge or lessons learned with other team members. Good practices allow organizations to maintain quality and increase the ability to deliver changes quickly.

My takeaway from this research is that change management is still important, but the traditional approach must evolve for larger organizations. The goal is not to remove all approval processes because some changes require additional reviews based on risk, security, or compliance requirements. The approval processes should not become a barrier that prevents the development process. The goal is to create a system where teams can safely make changes through automation, team collaboration, and provide continuous feedback. Reliability comes from building quality into the process rather than relying on the final approval. By using automation, telemetry, and testing, risks can be reduced while achieving the deployment speeds needed.

Resources:

Atlassian. (May 12, 2020). [Best practices for change management in the age of DevOps](#)

LaunchDarkly. (n.d.). [Reconciling change management and continuous delivery](#)

PLAT4MATION. (n.d.). [How to close the gap between DevOps and Change Management](#)